Skill-19

Name: k.vaishali

Roll no: 2520030117

Section: 8

Aim:

To register signal handlers, configure signal actions, handle interrupts, manage signal context, verify stability, and test signal events. Also, to capture SIGINT, forward signals, protect the shell process, terminate foreground jobs, update job states, and test interrupt handling.

Objective:

- To register signal handlers for different signals.
- To configure signal actions using sigaction().
- To handle interrupts using signal handlers.
- To capture the SIGINT signal generated by Ctrl+C.
- To forward signals to the foreground process.
- To protect the shell process from accidental termination.
- To terminate foreground jobs when an interrupt is received.
- To update and verify foreground job states.
- To test signal events and interrupt handling.

Problem Statement:

Design and implement a shell-style signal handling mechanism in C that registers signal handlers and manages interrupts. The implementation should capture SIGINT, protect the shell process, forward the signal to the foreground job, terminate the foreground process when required, update the job state, and verify correct interrupt handling.

Theory:

1. Signal

A signal is a software notification sent to a process to indicate that an event has occurred. Signals can be generated by the operating system, another process, or
the user.

2. Signal Handler

A signal handler is a function that is executed when a process receives a particular signal. It allows the program to perform a specific action when the signal occurs.

3. SIGINT

SIGINT is the interrupt signal commonly generated when the user presses Ctrl+C in a terminal. It is normally used to interrupt a running foreground process.

4. Signal Registration

The sigaction() system call is used to register and configure signal handlers. It provides more control over signal handling than the older signal() function.

5. Signal Forwarding

In a shell environment, the shell can receive an interrupt while a foreground job is running. Instead of terminating itself, the shell can forward SIGINT to the foreground child process using kill().

6. Shell Process Protection

The shell process can temporarily ignore SIGINT while a foreground job is executing. This prevents Ctrl+C from terminating the shell itself.

7. Foreground Job

A foreground job is a process currently being executed by the shell while the shell waits for it to complete. The shell can monitor its state using waitpid().

8. Process Termination

A process can terminate normally or because it received a signal. The parent process can determine how the child terminated using the status returned by waitpid().

9. Job State Management

The job state indicates whether the foreground process exited normally or was

terminated by a signal. Functions such as WIFEXITED() and WIFSIGNALED() can be used to examine the process status.

10. Signal Context

Signal handling requires maintaining information about the current foreground process so that an incoming interrupt can be directed to the correct process.

Algorithm:

1. Start the program.
2. Define the signal handler for SIGINT.
3. Configure the signal action using sigaction().
4. Create a child process using fork().
5. In the child process, restore the default SIGINT action.
6. Execute the foreground command.
7. In the parent process, protect the shell from SIGINT.
8. Store the PID of the foreground process.
9. Wait for the foreground process using waitpid().
10. If SIGINT is received, forward it to the foreground process using kill().
11. Check whether the foreground process exited normally or was terminated by a signal.
12. Update and display the job state.
13. Restore the original signal action.
14. Stop the program.

Data Used:

- Process ID (pid_t) to store the foreground process ID.
- struct sigaction to configure signal handling.
- Integer variable to store the child process status.
- SIGINT signal for interrupt handling.

Operations Used:

• sigaction() for registering and configuring signal handlers.

• signal() for restoring the default signal action in the child.

• fork() for creating the foreground child process.

• kill() for forwarding SIGINT.

• waitpid() for waiting for the foreground process. •

WIFEXITED() for checking normal termination. •

WIFSIGNALED() for checking signal-based termination.

• WTERMSIG() for identifying the terminating signal.

• WEXITSTATUS() for obtaining the exit status. **Program:**

```
#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <signal.h>

#include <sys/wait.h>


volatile sig_atomic_t fg_pid = -1;


void handle_sigint(int sig) {

 if (fg_pid > 0)

 kill(fg_pid, SIGINT);

}


 int main() {

 pid_t pid;

 int status;
```

```
 struct sigaction sa, old_sa;


 printf("Skill - 19: Signal Handling\n");

printf("----------------------------\n");
 sa.sa_handler = handle_sigint;

sigemptyset(&sa.sa_mask);

sa.sa_flags = 0;


 sigaction(SIGINT, &sa, &old_sa);
 pid = fork();


 if (pid < 0) {

 perror("fork");

 return 1;

 }


 if (pid == 0) {

 signal(SIGINT, SIG_DFL);


 printf("Foreground job started. Press Ctrl+C to interrupt.\n");

execlp("sleep", "sleep", "20", NULL);


 perror("sleep");

 exit(1);

 }
```

```
 fg_pid = pid;


 printf("Shell is protected. Foreground PID:
%d\n", pid); printf("Waiting for foreground
job...\n");
 waitpid(pid, &status, 0);

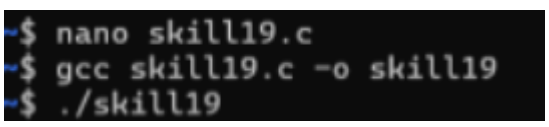

 fg_pid = -1;

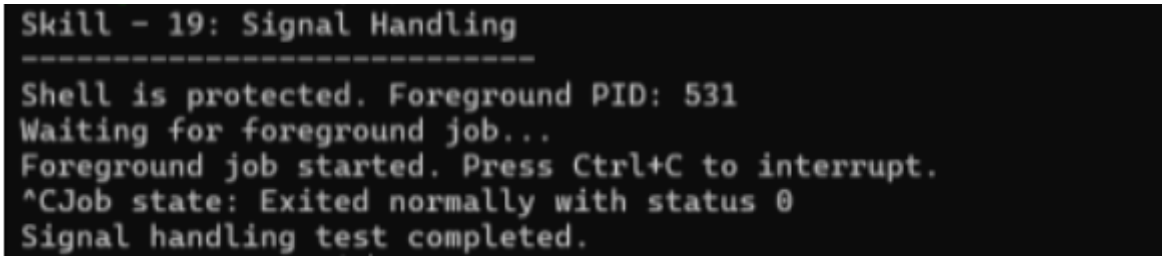

 sigaction(SIGINT, &old_sa, NULL);
 if (WIFEXITED(status)) {
 printf("Job state: Exited normally with status %d\n",
 WEXITSTATUS(status));
 } else if (WIFSIGNALED(status)) {
 printf("Job state: Terminated by signal %d\n",
 WTERMSIG(status));
 }


 printf("Signal handling test completed.\n");


 return 0;
}
```

Output:





Program Explanation
16.1 Signal Handler Registration

The sigaction() function registers handle_sigint() as the handler for SIGINT. When Ctrl+C generates SIGINT, the registered handler is executed.

16.2 Process Creation

The fork() system call creates a child process. The child process acts as the foreground job while the parent process acts as the shell.

16.3 SIGINT Handling
The handle_sigint() function checks whether a foreground process exists. If it exists, kill() sends SIGINT to that process.

16.4 Shell Protection

The parent shell uses the registered signal handler instead of being terminated directly by SIGINT. Therefore, the interrupt can be forwarded to the foreground job.

16.5 Foreground Job Execution

The child restores the default SIGINT behavior and executes the sleep command. Pressing Ctrl+C interrupts this foreground process.

16.6 Process Synchronization

The parent uses waitpid() to wait until the foreground process finishes or is

terminated. **16.7 Job State**

WIFEXITED() checks whether the child exited normally, while WIFSIGNALED() checks whether it was terminated by a signal. WTERMSIG() identifies the signal responsible for termination.

Testing:

- Tested signal handler registration using sigaction().

• Tested SIGINT using Ctrl+C.

• Verified that the shell process was protected.

• Verified that SIGINT was forwarded to the foreground process. •

Verified that the foreground process was terminated by the interrupt. •

Verified that the parent waited for the foreground process.

• Verified that the final job state was updated correctly.
• Tested normal completion of the foreground process.

Test Cases:

Test Case	Input / Action Expected Result
1	Run program without pressing Ctrl+C — Foreground job completes normally.

2 Press Ctrl+C while job is running SIGINT is sent to foreground job.

Test Case	Input / Action Expected Result
3	Press Ctrl+C during sleep Foreground job terminates.
4	Check parent shell Shell remains active.
5	Check process status Job state shows signal-based termination.
6	Run program until completion Job exits normally and status is displayed.

Observation:

• The signal handler was registered successfully.

• SIGINT was captured when Ctrl+C was pressed.

• The shell process remained protected while the foreground job was running.

• The interrupt was forwarded to the foreground process.

• The foreground job was terminated when SIGINT was

received. • The parent process successfully waited for the

foreground job.

• The job termination state was identified using process-status macros.

• Signal handling and interrupt forwarding worked successfully.

Result:

Signal handling was successfully implemented. The program captured SIGINT, protected the shell process, forwarded the signal to the foreground job, terminated the foreground process, and correctly identified its final job state.

Conclusion:

The practical demonstrated how operating systems handle signals and interrupts in a shell-like environment. It showed signal registration, SIGINT handling, signal forwarding, shell protection, foreground job control, process synchronization, and job-state management using C system calls.